

ESTÁNDARES WEB MODERNOS EN EL DESARROLLO FRONTEND: ARQUITECTURA, ACCESIBILIDAD Y APLICACIÓN PRÁCTICA EN CINEVERSE

Arias Moreira, Maybelin Gregoria; Castro Palma, Justyn Moises; Rios Cuyabazo, Jhon Kevin^{a,b,c*}

Resumen

El presente trabajo analiza los estándares web modernos aplicados al desarrollo frontend, abarcando arquitectura de aplicaciones web, HTML5 semántico, CSS3 avanzado y JavaScript ES6+. Se estudian conceptos fundamentales como la clasificación de aplicaciones (SPA, PWA, arquitecturas monolíticas y microservicios), el ciclo de vida de solicitudes HTTP/HTTPS, los principios de accesibilidad WCAG 2.1 y las técnicas de diseño inclusivo. Como componente práctico, se implementó la aplicación web CineVerse, un buscador de películas y series que consume la API pública TVMaze, aplicando diseño responsivo mobile-first, módulos ES6, programación asíncrona y persistencia con LocalStorage. La aplicación fue auditada mediante Lighthouse y WAVE, verificando el cumplimiento de criterios de accesibilidad, rendimiento y buenas prácticas. Los resultados demuestran que la integración de HTML5 semántico, CSS Grid/Flexbox y programación modular permite construir aplicaciones web eficientes, accesibles y compatibles con estándares internacionales de calidad.

Palabras clave: accesibilidad web, arquitectura frontend, CSS3, HTML5, JavaScript ES6+

Abstract

This paper analyzes modern web standards applied to frontend development, covering web application architecture, semantic HTML5, advanced CSS3, and JavaScript ES6+. Key concepts studied include application classification (SPA, PWA, monolithic and microservice architectures), the HTTP/HTTPS request lifecycle, WCAG 2.1 accessibility principles, and inclusive design techniques. As a practical component, the CineVerse web application was implemented — a movie and TV series search engine that consumes the TVMaze public API, applying mobile-first responsive design, ES6 modules, asynchronous programming, and LocalStorage persistence. The application was audited using Lighthouse and WAVE, verifying compliance with accessibility, performance, and best practice criteria. Results demonstrate that the integration of semantic HTML5, CSS Grid/Flexbox, and modular programming enables the construction of efficient, accessible web applications aligned with international quality standards.

Keywords: CSS3, frontend architecture, HTML5, JavaScript ES6+, web accessibility

I. INTRODUCCIÓN

La etapa final de un trabajo de investigación es comunicar los resultados, de manera que éstos permitan integrar los conocimientos a la comunidad científica y a la sociedad en general. Estos resultados deben tener validez, importancia, novedad y utilidad para la práctica profesional, así como para la comunidad científica en general. Para interpretar y comunicar los resultados de un estudio, los autores requiere experiencia, conocimiento de estadística, capacidad de análisis, creatividad, compenetración intelectual y razonamiento lógico. Esto permitirá la realización de comentarios pertinentes en lo referido a trabajos previos, así como el manejo adecuado de los datos logrados para interpretarlos y valorarlos.

En esta sección se debe justificar el trabajo de investigación y su relación con otros trabajos anteriores. Debe existir una amplia revisión bibliográfica, de tal manera de dejar claramente establecido el "estado del arte" en la temática abordada y las motivaciones que dieron origen al estudio que se va publicar. Las referencias citadas en el texto deben estar bien documentadas y actualizadas, y se debe evitar un número excesivo de citaciones. No se busca citar toda la información científica sobre el tema, sino lo más destacado y relevante. La introducción hace la apertura del artículo, ambienta y dirige al lector; asimismo, informa el tema a tratar, define el problema de investigación, presenta los antecedentes que fundamentan el estudio, define los objetivos, destaca el valor, el por qué y la utilidad del trabajo realizado.

* Correspondencia a: Nombre de la institución, universidad (si corresponde), dirección, ciudad, país. Teléfono: ++593 XXXXXXXX; fax: ++593 XXXXXXXX.
Correo electrónico: autor@mail.ec

El presente trabajo tiene como objetivo guiar a quienes se inician en la escritura científica, explicando aspectos referidos al contenido de los manuscritos y la mejor forma de presentar los resultados de la investigación científica.

II. OBJETIVOS

En esta sección se describen los métodos o técnicas empleadas en el desarrollo del trabajo de investigación. Además, deben incluirse los reactivos (marca y pureza), materiales y equipos (modelo y detalles técnicos importantes) utilizados. Se deben incluir parámetros usados en los equipos y cualquier aspecto que se juzgue necesario para que el lector del artículo pueda replicar la parte experimental y los resultados del trabajo de investigación. Resulta muy conveniente referirse a métodos utilizados por otros autores y/o trabajos previos, eso sí, siempre citando aquellos trabajos. Se debe tener en cuenta aspectos como el diseño experimental, indicando las variables, muestras y población tomados en base a los objetivos del estudio. Se debe explicar la dimensión temporal, el momento, número de veces y cualquier otro detalle referido a la recolección de la información. Finalmente es importante recordar que el diseño contribuye a la validez interna del estudio, además debe contener el escenario en el que se desarrolló la investigación, el o los sujetos, el tamaño muestral, condiciones de trabajo, métodos de recolección de las muestras y como fueron analizados los datos.

III. DESARROLLO TEÓRICO

En esta sección se describen los métodos o técnicas empleadas en el desarrollo del trabajo de investigación. Además, deben incluirse los reactivos (marca y pureza), materiales y equipos (modelo y detalles técnicos importantes) utilizados. Se deben incluir parámetros usados en los equipos y cualquier aspecto que se juzgue necesario para que el lector del artículo pueda replicar la parte experimental y los resultados del trabajo de investigación. Resulta muy conveniente referirse a métodos utilizados por otros autores y/o trabajos previos, eso sí, siempre citando aquellos trabajos. Se debe tener en cuenta aspectos como el diseño experimental, indicando las variables, muestras y población tomados en base a los objetivos del estudio. Se debe explicar la dimensión temporal, el momento, número de veces y cualquier otro detalle referido a la recolección de la información. Finalmente es importante recordar que el diseño contribuye a la validez interna del estudio, además debe contener el escenario en el que se desarrolló la investigación, el o los sujetos, el tamaño muestral, condiciones de trabajo, métodos de recolección de las muestras y como fueron analizados los datos.

3.1 Arquitectura de Aplicaciones Web y Diseño Inclusivo

Durante los últimos años, la arquitectura de las aplicaciones web ha evolucionado desde modelos estáticos simples hacia ecosistemas distribuidos capaces de ofrecer interacción en tiempo real, procesamiento asíncrono y experiencias adaptables a distintos dispositivos. Esta evolución no solo responde a necesidades funcionales, sino también a exigencias relacionadas con rendimiento, accesibilidad, seguridad y mantenibilidad. En el contexto del desarrollo frontend moderno, comprender la estructura arquitectónica de una aplicación permite tomar decisiones técnicas más adecuadas respecto al renderizado, la comunicación cliente-servidor y la experiencia de usuario.

3.1.1 Taxonomía formal de aplicaciones web

La clasificación de aplicaciones web puede realizarse considerando criterios como el tipo de renderizado, la distribución de componentes, el comportamiento del cliente y la forma en que se gestionan los datos. Las aplicaciones estáticas representan el modelo más simple, ya que el servidor entrega archivos HTML, CSS y JavaScript sin procesamiento dinámico. Este enfoque posee bajo costo de infraestructura y tiempos de carga reducidos; sin embargo, presenta limitaciones cuando se requieren interacciones complejas o personalización de contenido.

Por otro lado, las aplicaciones dinámicas generan contenido en función de solicitudes específicas del usuario. Plataformas como Facebook o Amazon utilizan procesamiento dinámico para personalizar recomendaciones, sesiones y contenido interactivo. Este tipo de arquitectura incrementa la dependencia del servidor y exige mayor control sobre bases de datos, autenticación y manejo de sesiones.

Las SPA (Single Page Applications) modifican el modelo tradicional de navegación debido a que gran parte del procesamiento ocurre en el cliente utilizando JavaScript. Frameworks como React o Vue permiten actualizar únicamente partes del DOM sin recargar toda la página. Aunque esto mejora la fluidez de navegación, también introduce desafíos relacionados con SEO, accesibilidad y rendimiento inicial.

En contraste, las PWA (Progressive Web Applications) buscan combinar características web y móviles mediante Service Workers, almacenamiento local y funcionamiento offline. Tecnologías como caché inteligente y sincronización en segundo plano permiten que aplicaciones web funcionen incluso con conectividad limitada.

Tabla 1. Comparación técnica entre SPA y PWA

Criterio	SPA	PWA
Renderizado	Principalmente del lado cliente	Puede combinar renderizado cliente y capacidades progresivas
Funcionamiento offline	Limitado	Sí, mediante Service Workers
SEO	Puede verse afectado	Mejorable mediante SSR y optimización
Experiencia de usuario	Navegación fluida	Similar a aplicaciones móviles
Instalación	Generalmente no	Sí, instalable desde navegador
Complejidad inicial	Media	Alta
Dependencia de JavaScript	Elevada	Elevada
Uso empresarial	Dashboards, paneles administrativos	Apps móviles web progresivas

Además, las arquitecturas monolíticas agrupan frontend, backend y acceso a datos en un único sistema centralizado. Aunque simplifican el despliegue inicial, presentan dificultades de escalabilidad conforme el sistema crece. Por el contrario, los microservicios dividen funcionalidades independientes que pueden desplegarse y mantenerse de manera separada. Empresas como [Netflix](#) utilizan microservicios debido a la necesidad de alta disponibilidad y escalabilidad distribuida.

Desde una perspectiva crítica, el uso de SPA puede mejorar significativamente la experiencia visual, pero también puede perjudicar la accesibilidad cuando el contenido dinámico no es correctamente interpretado por lectores de pantalla o cuando la navegación depende exclusivamente de JavaScript. Por esta razón, muchas arquitecturas modernas combinan SSR (Server Side Rendering) y técnicas híbridas para equilibrar rendimiento y accesibilidad.

3.1.2 Ciclo de vida de una solicitud HTTP/HTTPS

Cuando un usuario ingresa una URL en el navegador, se inicia una secuencia de procesos que involucra protocolos de red, mecanismos de seguridad y renderizado frontend. El primer paso corresponde a la resolución DNS, donde el navegador consulta la dirección IP asociada al dominio solicitado. Posteriormente se establece una conexión TCP y, si el sitio utiliza HTTPS, se ejecuta un TLS Handshake para cifrar la comunicación entre cliente y servidor.

Una vez establecida la conexión segura, el navegador envía una solicitud HTTP utilizando métodos específicos según la operación requerida. El método GET se emplea para recuperar información sin modificar recursos; POST se utiliza para crear datos nuevos; PUT reemplaza completamente un recurso existente; PATCH realiza modificaciones parciales; y DELETE elimina información almacenada.

Tabla 2. Métodos HTTP y su aplicación arquitectónica

Método	Función principal	Uso frecuente en frontend
GET	Obtener datos	Consultas a APIs
POST	Crear recursos	Formularios y registros
PUT	Actualizar completamente	Edición total de registros
PATCH	Actualización parcial	Cambios específicos
DELETE	Eliminar recursos	Eliminación de registros

Las respuestas HTTP incluyen códigos de estado que permiten interpretar el resultado de la petición. Los códigos 2xx indican éxito; 3xx redirecciones; 4xx errores del cliente; y 5xx errores del servidor.

Tabla 3. Clasificación de códigos HTTP

Categoría	Significado
2xx	Operación exitosa
3xx	Redireccionamiento
4xx	Error del cliente
5xx	Error interno del servidor

Además del intercambio de datos, las cabeceras de seguridad cumplen un rol importante dentro de la arquitectura web moderna. HSTS obliga el uso de HTTPS; CSP reduce riesgos de inyección de scripts maliciosos; y X-Frame-Options evita ataques de clickjacking mediante iframes externos.

En aplicaciones frontend modernas, la selección adecuada de métodos HTTP influye directamente en la organización del código cliente, especialmente en arquitecturas basadas en consumo de APIs REST. Un frontend desacoplado requiere manejar asincronía, validaciones y errores de red de manera explícita para evitar fallos de experiencia de usuario.

3.1.3 Principios WCAG 2.1 aplicados al diseño inclusivo

Las pautas WCAG 2.1 desarrolladas por W3C establecen estándares internacionales para garantizar accesibilidad digital [1]. Estas directrices se estructuran sobre cuatro principios fundamentales conocidos como POUR: Perceptible, Operable, Comprensible y Robusto.

El principio Perceptible establece que la información debe poder identificarse mediante distintos canales sensoriales. Por ejemplo, los criterios de nivel AA exigen contraste mínimo de 4.5:1 entre texto y fondo, así como alternativas textuales para contenido multimedia.

El principio Operable requiere que todos los componentes puedan utilizarse mediante teclado sin depender exclusivamente del mouse. Esto implica navegación secuencial lógica, visibilidad del foco y tiempos de interacción adecuados.

Comprensible se relaciona con interfaces previsibles y mensajes claros. Formularios accesibles deben mostrar errores comprensibles y etiquetas correctamente asociadas. Finalmente, Robusto busca compatibilidad con tecnologías asistivas como NVDA y JAWS mediante HTML semántico y atributos ARIA válidos.

Tabla 4. Principios POUR y aplicación práctica

Principio	Aplicación en el proyecto
Perceptible	Contraste AA y texto alternativo
Operable	Navegación completa por teclado
Comprensible	Mensajes de error claros
Robusto	Compatibilidad con lectores de pantalla

En Ecuador, la accesibilidad digital posee implicaciones legales vinculadas con la Ley Orgánica de Discapacidades y estándares técnicos como la Norma INEN 2620, los cuales promueven igualdad de acceso a servicios digitales públicos y privados [1].

Para evaluar accesibilidad, existen herramientas complementarias como [WAVE Web Accessibility Evaluation Tool](#), [Lighthouse](#) y [axe DevTools](#).

Tabla 5. Comparación de herramientas de auditoría

Herramienta	Detecta	Limitaciones
WAVE	Problemas visuales y semánticos	No analiza lógica compleja
Lighthouse	Rendimiento, SEO y accesibilidad	Menor profundidad manual
axe DevTools	Validaciones WCAG avanzadas	Requiere interpretación técnica

La combinación de herramientas resulta necesaria porque ninguna puede identificar completamente problemas relacionados con contexto, navegación real o interpretación humana.

3.1.4 Análisis crítico sobre SPA y accesibilidad

Aunque las SPA mejoran la interacción dinámica y reducen recargas completas de página, también pueden afectar negativamente la accesibilidad cuando el contenido cambia sin notificación adecuada a tecnologías asistidas. En aplicaciones altamente dinámicas, los lectores de pantalla pueden no detectar actualizaciones del DOM si no se utilizan regiones aria-live o manejo explícito del foco. Otro problema frecuente ocurre cuando la navegación depende completamente de JavaScript. Si el contenido principal no existe inicialmente en HTML, motores de búsqueda y tecnologías asistidas pueden interpretar páginas vacías durante el renderizado inicial.

Para mitigar estos problemas, las técnicas de progressive enhancement permiten construir primero una base funcional accesible y posteriormente añadir características avanzadas mediante JavaScript. Estrategias como SSR, hidratación parcial y renderizado híbrido ayudan a mejorar SEO, accesibilidad y tiempos de carga sin sacrificar experiencia interactiva.

En el contexto del proyecto frontend del equipo, aplicar principios de accesibilidad desde la etapa inicial evita depender posteriormente de correcciones costosas. La combinación entre HTML semántico, diseño responsive y validaciones WCAG permite desarrollar aplicaciones más inclusivas, mantenibles y compatibles con estándares web actuales.

3.2 HTML5 Semántico y Formularios Avanzados

El desarrollo frontend moderno ya no se limita únicamente a la construcción visual de interfaces web. Actualmente, tecnologías como HTML5 cumplen un papel fundamental en accesibilidad, posicionamiento SEO, interoperabilidad entre navegadores y compatibilidad con tecnologías asistivas [2], [3]. La incorporación de semántica estructural y formularios accesibles permite que las aplicaciones web sean interpretadas correctamente tanto por motores de búsqueda como por usuarios que utilizan lectores de pantalla o métodos alternativos de navegación. En consecuencia, HTML5 se ha convertido en un componente esencial dentro de arquitecturas frontend centradas en experiencia de usuario y diseño inclusivo.

3.2.1 Semántica estructural y organización del contenido

La semántica en HTML5 consiste en utilizar etiquetas que describen correctamente el propósito del contenido dentro del documento. Esta práctica mejora la interpretación del DOM por parte de navegadores, motores de búsqueda y tecnologías asistivas como NVDA y JAWS. Google, por ejemplo, utiliza información semántica derivada del árbol de accesibilidad para comprender la jerarquía y relevancia del contenido, lo que influye directamente en estrategias SEO [2].

Uno de los problemas más frecuentes en proyectos frontend iniciales es el uso excesivo de <div> para estructurar toda la interfaz. Aunque <div> funciona como un contenedor genérico, carece de significado semántico, por lo que no proporciona contexto sobre el contenido que encapsula. HTML5 introdujo etiquetas semánticas que permiten organizar mejor la información y mejorar accesibilidad.

3.2.2 WAI-ARIA y accesibilidad avanzada

3.2.3 Formularios HTML5 y experiencia de usuario accesible

3.2.4 APIs HTML5 para almacenamiento local

Tabla 1. Comparación semántica de etiquetxas estructurales

Etiqueta	Uso correcto	Característica principal
<article>	Contenido independiente y reutilizable	Puede existir fuera del contexto principal
<section>	Agrupación temática de contenido	Requiere encabezado relacionado
<div>	Contenedor genérico sin significado semántico	Uso únicamente estructural
<main>	Contenido principal único de la página	Solo debe existir uno
<aside>	Información complementaria o relacionada	Puede separarse del contenido central

El elemento <article> se utiliza para contenido autónomo como noticias, publicaciones o tarjetas de productos. En cambio, <section> agrupa bloques temáticos relacionados dentro de una misma página. Utilizar incorrectamente estas etiquetas puede generar problemas de interpretación para lectores de pantalla y motores de indexación.

Asimismo, HTML5 establece que debe existir un único <main> por documento, debido a que representa el contenido principal de la interfaz. El uso de múltiples elementos <main> puede romper la navegación asistiva. De igual manera, <aside> debe contener información secundaria como anuncios, recomendaciones o enlaces complementarios.

Otro aspecto importante es la jerarquía correcta de encabezados <h1> a <h6>. Muchos desarrolladores utilizan encabezados únicamente por razones visuales, ignorando que forman una estructura lógica de navegación para lectores de pantalla. Saltar directamente de <h1> a <h4> puede generar pérdida de contexto semántico y dificultades de comprensión para usuarios con discapacidad visual.

- WAI-ARIA y accesibilidad avanzada

WAI-ARIA (Web Accessibility Initiative – Accessible Rich Internet Applications) permite complementar accesibilidad en componentes dinámicos que HTML por sí solo no puede describir adecuadamente [6]. Sin embargo, uno de los principios más importantes dentro de accesibilidad web establece que “no ARIA is better than bad ARIA”, debido a que una implementación incorrecta puede empeorar la experiencia de navegación asistiva.

ARIA no debe reemplazar HTML semántico. Por ejemplo, utilizar <button> correctamente es preferible a usar un <div role="button">, ya que los elementos nativos ya poseen comportamiento accesible integrado.

- Tabla 2. Roles ARIA fundamentales

Rol	Función
role="navigation"	Define áreas de navegación
role="dialog"	Identifica ventanas modales
role="alert"	Notifica mensajes importantes inmediatos
role="status"	Comunica cambios dinámicos no críticos

Además de roles, ARIA incorpora estados y propiedades que permiten describir comportamientos dinámicos.

- Tabla 3. Estados y propiedades ARIA

Atributo	Función
aria-expanded	Indica si un elemento desplegable está abierto
aria-selected	Marca elementos seleccionados

Atributo	Función
aria-checked	Representa estados de selección
aria-label	Define etiquetas accesibles
aria-labelledby	Relaciona etiquetas visibles
aria-describedby	Añade descripciones adicionales
aria-live	Anuncia contenido dinámico automáticamente

En aplicaciones SPA modernas, atributos como aria-live resultan importantes debido a que el contenido puede actualizarse sin recargar la página. Si estas actualizaciones no son notificadas, los lectores de pantalla podrían no detectar cambios importantes dentro de la interfaz.

- Formularios HTML5 y experiencia de usuario accesible

Los formularios representan uno de los componentes más sensibles dentro de accesibilidad web, debido a que constituyen el principal mecanismo de interacción entre usuario y sistema. HTML5 incorporó múltiples tipos de `<input>` diseñados para mejorar experiencia de usuario y validación nativa.

- Tabla 4. Tipos de input más utilizados y su impacto UX

Tipo	Función	Beneficio en móviles
email	Correos electrónicos	Activa teclado con “@”
tel	Números telefónicos	Abre teclado numérico
date	Fechas	Selector nativo
number	Valores numéricos	Mejora entrada de datos
password	Contraseñas	Ocultar caracteres
search	Búsquedas	Optimización UX

La validación nativa de HTML5 ofrece ventajas importantes debido a que funciona incluso sin JavaScript y mantiene compatibilidad con tecnologías asistivas. Atributos como `required`, `minlength`, `maxlength` y `pattern` permiten restringir entradas inválidas de forma automática.

No obstante, la validación nativa también presenta limitaciones. Los mensajes de error cambian entre navegadores y poseen poca capacidad de personalización visual. Por esta razón, muchos sistemas implementan validación accesible mediante JavaScript complementario.

Ejemplo de validación accesible con JavaScript

```
<form id="registro">
  <label for="correo">Correo electrónico</label>
  <input type="email" id="correo" required>
  <span id="errorCorreo" aria-live="polite"></span>

  <button type="submit">Enviar</button>
</form>
```

```
<script>
const formulario = document.getElementById("registro");
```

```
formulario.addEventListener("submit", (e) => {
  const correo = document.getElementById("correo");
  const error = document.getElementById("errorCorreo");
```

```
  if (!correo.value.includes("@")) {
    e.preventDefault();
    error.textContent = "Ingresa un correo válido.";
    correo.focus();
  }
});
</script>
```

Este enfoque mejora accesibilidad debido a que:
no bloquea innecesariamente datos válidos,

mantiene foco en el campo problemático,
y utiliza aria-live para anunciar errores dinámicos.

- APIs HTML5 para almacenamiento local

HTML5 introdujo mecanismos de almacenamiento cliente que permiten conservar información sin depender completamente del servidor. Los más utilizados son localStorage, sessionStorage y cookies.

- Tabla 5. Comparación entre mecanismos de almacenamiento

Característica	localStorage	sessionStorage	Cookies
Persistencia	Permanente	Hasta cerrar pestaña	Configurable
Capacidad aproximada	5 MB	5 MB	4 KB
Acceso JavaScript	Sí	Sí	Sí
Envío automático al servidor	No	No	Sí
Riesgo XSS	Alto	Alto	Medio
Uso recomendado	Preferencias y temas	Sesiones temporales	Autenticación

localStorage resulta útil para almacenar preferencias persistentes como modo oscuro o configuraciones de usuario. sessionStorage se utiliza principalmente para datos temporales relacionados con la sesión activa. En cambio, las cookies continúan siendo necesarias para autenticación y manejo de sesiones debido a que pueden enviarse automáticamente al servidor mediante cabeceras HTTP.

Sin embargo, almacenar información sensible directamente en localStorage representa un riesgo de seguridad importante frente a ataques XSS, debido a que JavaScript puede acceder fácilmente a los datos almacenados. Por esta razón, aplicaciones modernas suelen utilizar cookies seguras con atributos HttpOnly y Secure.

En el contexto del proyecto frontend desarrollado por el equipo, el uso de almacenamiento local puede mejorar rendimiento y experiencia de usuario al conservar configuraciones visuales, filtros o preferencias sin requerir solicitudes constantes al servidor.

3.3 CSS3 Avanzado: Grid, Flexbox y Responsividad

3.3.1 CSS Grid vs Flexbox

CSS3 incorporó dos modelos de diseño fundamentales para la construcción de interfaces web modernas: CSS Grid Layout y Flexbox. Ambos sistemas permiten organizar elementos visuales dentro de una página web de manera eficiente, aunque fueron diseñados para resolver problemas distintos relacionados con la distribución y alineación de contenido. Actualmente constituyen las tecnologías más utilizadas para la creación de interfaces responsivas debido a su flexibilidad y compatibilidad con los navegadores modernos [8].

CSS Grid es un sistema de diseño bidimensional que permite controlar simultáneamente filas y columnas dentro de un contenedor. Gracias a propiedades como grid-template-columns, grid-template-rows, grid-template-areas y gap, los desarrolladores pueden construir estructuras complejas manteniendo una organización clara y escalable. Este modelo resulta especialmente útil para diseñar layouts principales, paneles administrativos, galerías y aplicaciones web con múltiples secciones distribuidas en una cuadrícula [8].

Por otro lado, Flexbox fue diseñado como un sistema unidimensional orientado a la distribución de elementos a lo largo de un único eje principal, ya sea horizontal o vertical. Sus propiedades permiten alinear, distribuir y reorganizar componentes dinámicamente según el espacio disponible. Debido a estas características, Flexbox es ampliamente utilizado para menús de navegación, formularios, barras de herramientas y componentes internos de una interfaz [5].

Tabla 6. Comparación técnica entre CSS Grid y Flexbox

Criterio	CSS Grid	Flexbox
Modelo de diseño	Bidimensional	Unidimensional
Control de filas	Sí	No
Control de columnas	Sí	No
Diseño complejo	Excelente	Limitado
Componentes individuales	Bueno	Excelente
Distribución automática	Sí	Sí
Gap nativo	Sí	Sí
Responsividad	Muy alta	Alta

Dentro del proyecto CineVerse se utilizó CSS Grid para organizar la visualización de las películas obtenidas desde la API, permitiendo distribuir las tarjetas dinámicamente según el tamaño disponible de la pantalla.

Código 9. Implementación de Grid utilizada en el proyecto

```
movies-grid{

    display:grid;

    grid-template-columns:
        repeat(auto-fit,minmax(260px,1fr));

    gap:30px;
}
```

Esta implementación permite que las tarjetas se adapten automáticamente a diferentes resoluciones manteniendo una distribución uniforme y optimizando el aprovechamiento del espacio disponible.

3.3.2 Sistema responsive mobile-first

El crecimiento sostenido del acceso a Internet mediante dispositivos móviles ha impulsado la adopción de metodologías de diseño orientadas inicialmente a pantallas pequeñas. Entre ellas destaca Mobile-First, una estrategia que consiste en desarrollar primero para dispositivos móviles y posteriormente ampliar las funcionalidades mediante media queries para resoluciones superiores [9]. Este enfoque ofrece múltiples ventajas. En primer lugar, permite optimizar el rendimiento al cargar únicamente los estilos esenciales durante la primera visualización. Además, favorece una mejor experiencia de usuario debido a que obliga a priorizar el contenido más importante dentro de espacios reducidos. Finalmente, facilita el mantenimiento del código CSS al estructurar los estilos de forma progresiva [9].

Los puntos de ruptura o breakpoints más utilizados actualmente se muestran en la siguiente tabla.

Tabla 7. Breakpoints recomendados para diseño responsive

Dispositivo	Resolución aproximada
Smartphones	320px – 767px
Tablets	768px – 1023px
Laptops	1024px – 1439px
Escritorio	1440px o superior

Dentro del proyecto CineVerse se implementaron Media Queries para adaptar la interfaz a diferentes tamaños de pantalla.

Código 10. Media Query utilizada en el proyecto

```
@media(min-width:768px){

    .search-form{
        grid-template-columns:1fr 1fr;
    }
}
```

Mediante esta configuración, el formulario de búsqueda modifica automáticamente su distribución cuando la aplicación es visualizada desde dispositivos con resoluciones superiores a 768 píxeles, mejorando la experiencia de navegación y aprovechando mejor el espacio disponible.

Otra técnica moderna utilizada en el desarrollo responsive corresponde al empleo de unidades relativas y funciones adaptativas como clamp(), las cuales permiten que elementos como la tipografía se ajusten dinámicamente según el tamaño de la pantalla [8].

3.3.3 Variables CSS y sistemas de temas

Las Variables CSS, también conocidas como Custom Properties, representan una de las características más importantes incorporadas en las versiones modernas de CSS. Su principal objetivo consiste en permitir la reutilización de valores dentro de la hoja de estilos, facilitando el mantenimiento y la consistencia visual de una aplicación web [8].

Generalmente las variables se definen dentro del selector :root, lo que permite su acceso global desde cualquier componente de la

interfaz.

Código 11. Variables CSS implementadas en CineVerse

```
:root{

  --color-bg:#f5f1eb;
  --color-card:#ffffff;

  --color-primary:#c8a97e;
  --color-secondary:#8b7355;

  --color-text:#2e2e2e;
}
```

Posteriormente estos valores pueden reutilizarse mediante la función var().

```
body{

  background:var(--color-bg);

  color:var(--color-text);
}
```

El uso de variables CSS reduce significativamente la duplicación de código y simplifica futuras modificaciones de diseño. Si se requiere cambiar la identidad visual de la aplicación, basta con modificar los valores definidos en el selector raíz para que los cambios se propaguen automáticamente a todos los componentes asociados [8].

Además, las variables CSS facilitan la implementación de sistemas de temas claros y oscuros. En el proyecto desarrollado se incorporó soporte para modo oscuro mediante la media query prefers-color-scheme.

Código 12. Implementación del tema oscuro

```
@media(prefers-color-scheme:dark){

  :root{

    --color-bg:#161616;
    --color-card:#242424;

    --color-text:#ffffff;
  }
}
```

Esta funcionalidad permite adaptar automáticamente la apariencia de la aplicación según la configuración establecida por el usuario en su sistema operativo, mejorando la experiencia de uso y reduciendo la fatiga visual.

3.3.4 Accesibilidad aplicada a CSS

La accesibilidad web constituye un conjunto de prácticas orientadas a garantizar que los contenidos digitales puedan ser utilizados por todas las personas, incluyendo usuarios con discapacidades visuales, auditivas, cognitivas o motoras. Aunque gran parte de la accesibilidad depende del uso correcto de HTML semántico y atributos WAI-ARIA, CSS también desempeña un papel fundamental dentro de este proceso [1].

Uno de los criterios más importantes definidos por las WCAG 2.1 corresponde al contraste mínimo entre texto y fondo. Para cumplir con el nivel AA, la relación de contraste debe ser al menos de 4.5:1 para textos normales [1].

Otro aspecto relevante consiste en mantener indicadores visibles de foco para usuarios que navegan mediante teclado. Cuando un

elemento recibe foco, este debe presentar una señal visual clara que permita identificarlo fácilmente.

Código 13. Ejemplo de indicador de foco

```
button:focus{

    outline:3px solid #c8a97e;
}
```

Asimismo, CSS permite respetar las preferencias relacionadas con la reducción de movimiento establecidas por el usuario mediante la media query prefers-reduced-motion.

```
@media(prefers-reduced-motion:reduce){

    *{

        animation:none !important;

        transition:none !important;
    }
}
```

Esta configuración beneficia especialmente a personas que presentan trastornos vestibulares o sensibilidad a las animaciones intensas [1].

Dentro del proyecto CineVerse también se implementaron colores con suficiente contraste y estructuras visuales claras para facilitar la lectura de la información presentada en las tarjetas de películas. La combinación entre diseño responsivo, buenas prácticas de contraste y compatibilidad con tecnologías asistivas contribuye a mejorar significativamente la accesibilidad general de la aplicación web [1], [8].

3.4 JavaScript ES6+ y Programación Asíncrona

3.4.1 Event Loop y asincronía

JavaScript fue diseñado como un lenguaje de ejecución single-threaded, lo que significa que únicamente puede ejecutar una tarea a la vez dentro del hilo principal [10]. Sin embargo, las aplicaciones web modernas requieren realizar múltiples operaciones simultáneamente, como solicitudes HTTP, temporizadores, lectura de archivos y actualización dinámica de interfaces. Para resolver esta limitación, JavaScript implementa un mecanismo denominado Event Loop, encargado de coordinar la ejecución de tareas asíncronas sin bloquear la interacción del usuario [10].

El funcionamiento del Event Loop se basa en varios componentes fundamentales. El primero es el Call Stack, donde se almacenan las funciones que se encuentran en ejecución. Cuando una operación requiere tiempo adicional, como una solicitud a una API externa, esta tarea es delegada a las Web APIs del navegador. Una vez completada, la respuesta se coloca en una cola de ejecución para que posteriormente el Event Loop la procese cuando el hilo principal se encuentre disponible [10].

La introducción de Promises en ECMAScript 6 permitió simplificar considerablemente el manejo de tareas asíncronas, eliminando gran parte de los problemas asociados al uso excesivo de callbacks. Posteriormente, las palabras reservadas async y await proporcionaron una sintaxis más clara y fácil de mantener.

Dentro del proyecto CineVerse se implementó programación asíncrona para realizar consultas a la API TVMaze sin bloquear la interfaz del usuario.

Código 14. Función asíncrona implementada en el proyecto

```
export async function fetchMovies(movieName){
    try{
        const response = await fetch(
            `https://api.tvmaze.com/search/shows?q=${movieName}`
        );

        if(!response.ok){
```

```

        throw new Error('Error al obtener películas');
    }

    const data = await response.json();

    return data;

} catch(error){

    console.error(error);

    return [];

}
}

```

Gracias al uso de `async` y `await`, la aplicación puede continuar respondiendo a las acciones del usuario mientras se obtiene la información desde el servidor remoto, mejorando significativamente la experiencia de navegación [10].

3.4.2 Módulos ES6 y arquitectura frontend

A medida que las aplicaciones web incrementan su tamaño y complejidad, resulta indispensable dividir el código en componentes independientes que faciliten su mantenimiento y reutilización. Antes de ECMAScript 6, esta separación se realizaba mediante patrones como IIFE o CommonJS. Sin embargo, ES6 incorporó un sistema de módulos nativo basado en las instrucciones `import` y `export`, convirtiéndose en el estándar actual para la organización del código JavaScript [10] [10].

Los módulos permiten encapsular funcionalidades específicas dentro de archivos independientes, reduciendo el acoplamiento entre componentes y favoreciendo la escalabilidad del sistema. Cada módulo puede exportar funciones, objetos o variables que posteriormente son reutilizadas por otros archivos de la aplicación.

En el proyecto CineVerse se implementó una arquitectura modular compuesta por tres archivos principales.

Tabla 8. Arquitectura modular implementada

Archivo	Función
api.js	Comunicación con la API TVMaze
ui.js	Generación dinámica de componentes visuales
app.js	Control principal de la aplicación

La comunicación entre módulos se realiza mediante importaciones ES6.

Código 15. Importación de módulos utilizada

```

import { fetchMovies } from './api.js';
import { renderMovies } from './ui.js';

```

Por su parte, el archivo encargado de la comunicación con la API exporta la función correspondiente.

```

export async function fetchMovies(movieName){
    ...
}

```

Esta arquitectura permite mantener una clara separación de responsabilidades dentro del proyecto, facilitando futuras modificaciones y mejorando la calidad general del software desarrollado [10].

3.4.3 Manejo robusto de errores con Fetch API

La Fetch API constituye actualmente el mecanismo estándar para realizar solicitudes HTTP desde aplicaciones frontend modernas [11]. Su principal ventaja radica en proporcionar una interfaz simple basada en Promises, permitiendo ejecutar operaciones de red de manera eficiente y compatible con los estándares actuales del navegador [11].

No obstante, toda comunicación con servicios externos puede verse afectada por diversos factores, entre ellos problemas de conectividad, respuestas inválidas, errores internos del servidor o restricciones de seguridad. Por esta razón, resulta fundamental implementar mecanismos adecuados para el manejo de excepciones.

Dentro del proyecto CineVerse se utilizó una estructura basada en bloques `try-catch` para capturar posibles errores producidos durante la comunicación con la API TVMaze.

Código 16. Manejo de errores implementado

```

try{
  const response = await fetch(
    `https://api.tvmaze.com/search/shows?q=${movieName}`
  );
  if(!response.ok){
    throw new Error('Error al obtener películas');
  }
  const data = await response.json();
  return data;
}catch(error){
  console.error(error);
  return [];
}

```

La propiedad `response.ok` permite verificar si la respuesta recibida pertenece al rango exitoso de códigos HTTP (200-299). En caso contrario, se genera una excepción personalizada mediante la instrucción `throw`.

Este enfoque mejora considerablemente la robustez de la aplicación, evitando interrupciones inesperadas y proporcionando mecanismos adecuados para gestionar situaciones de error durante la ejecución [11].

Además, cuando ocurre una excepción, la función devuelve un arreglo vacío para garantizar que la interfaz continúe funcionando correctamente y pueda informar al usuario sobre la ausencia de resultados.

3.4.4 Accesibilidad dinámica con JavaScript

Las aplicaciones web modernas actualizan constantemente el contenido de la página sin necesidad de realizar recargas completas. Aunque esta característica mejora la experiencia de usuario, también puede generar dificultades para personas que utilizan tecnologías asistivas como lectores de pantalla [1].

Para resolver este problema, las recomendaciones de accesibilidad establecen que los cambios dinámicos deben ser comunicados adecuadamente mediante atributos ARIA y mecanismos de gestión del foco [6].

Dentro del proyecto CineVerse se implementó una región dinámica que informa al usuario cuando se está realizando una búsqueda.

Código 17. Región accesible utilizada durante las búsquedas

```

<div id="loader"
  class="loader hidden"
  aria-live="polite">

```

Buscando películas...

```

</div>

```

El atributo `aria-live="polite"` permite que los lectores de pantalla anuncien automáticamente los cambios producidos dentro del elemento sin interrumpir otras actividades realizadas por el usuario [6].

Asimismo, JavaScript controla la visualización de mensajes informativos durante el proceso de búsqueda.

Código 18. Gestión dinámica de estados

```

loader.classList.remove('hidden');
empty.classList.add('hidden');

const movies = await fetchMovies(movieName);

loader.classList.add('hidden');
if(movies.length === 0){
  empty.classList.remove('hidden');
  moviesContainer.innerHTML = "";
  return;
}

```

```
}
```

Otra práctica relevante corresponde a la utilización de texto alternativo en las imágenes generadas dinámicamente.

```
</img>
```

La incorporación de estas técnicas permite que la información mostrada por la aplicación sea accesible para un mayor número de usuarios, contribuyendo al cumplimiento de los principios de percepción, operabilidad y comprensión definidos por las WCAG 2.1 [1], [6], [7].

IV. DESARROLLO PRÁCTICO

En esta sección se describen los métodos o técnicas empleadas en el desarrollo del trabajo de investigación. Además, deben incluirse los reactivos (marca y pureza), materiales y equipos (modelo y detalles técnicos importantes) utilizados. Se deben incluir parámetros usados en los equipos y cualquier aspecto que se juzgue necesario para que el lector del artículo pueda replicar la parte experimental y los resultados del trabajo de investigación. Resulta muy conveniente referirse a métodos utilizados por otros autores y/o trabajos previos, eso sí, siempre citando aquellos trabajos. Se debe tener en cuenta aspectos como el diseño experimental, indicando las variables, muestras y población tomados en base a los objetivos del estudio. Se debe explicar la dimensión temporal, el momento, número de veces y cualquier otro detalle referido a la recolección de la información. Finalmente es importante recordar que el diseño contribuye a la validez interna del estudio, además debe contener el escenario en el que se desarrolló la investigación, el o los sujetos, el tamaño muestral, condiciones de trabajo, métodos de recolección de las muestras y como fueron analizados los datos.

4.1 Descripción de la aplicación web

Como parte del desarrollo práctico del presente proyecto se implementó una aplicación web denominada CineVerse, orientada a la búsqueda y consulta de información relacionada con películas y series mediante el consumo de una API pública. La aplicación permite que los usuarios ingresen el nombre de una película o serie, consulten información actualizada desde un servicio externo y visualicen los resultados de manera organizada mediante tarjetas dinámicas.

El sistema fue desarrollado utilizando HTML5, CSS3 y JavaScript ES6+, aplicando los estándares web estudiados durante el curso. Además, se incorporaron principios de accesibilidad, diseño responsivo y programación modular para garantizar una experiencia de usuario adecuada en diferentes dispositivos.

Entre las principales funcionalidades implementadas se encuentran:

- Búsqueda de películas y series mediante palabras clave.
- Consumo de información desde una API externa.
- Generación dinámica de resultados.
- Persistencia de datos mediante LocalStorage.
- Diseño adaptable para dispositivos móviles y escritorio.
- Indicadores visuales de carga y mensajes informativos.

La aplicación consume información desde la API TVMaze, permitiendo acceder a datos actualizados relacionados con producciones audiovisuales de manera rápida y eficiente.

4.2 Tecnologías utilizadas

El desarrollo de CineVerse requirió la integración de diversas tecnologías frontend modernas que permitieron construir una aplicación dinámica, modular y accesible.

Tabla 9. Tecnologías utilizadas en el proyecto

Tecnología	Función
HTML5	Estructura semántica de la aplicación
CSS3	Diseño visual y responsividad
JavaScript ES6+	Programación de la lógica de negocio
Fetch API	Comunicación con servicios externos
LocalStorage	Persistencia de información local
TVMaze API	Fuente de información de películas
Visual Studio Code	Entorno de desarrollo
Lighthouse	Auditoría de calidad web
WAVE	Evaluación de accesibilidad

La integración de estas herramientas permitió desarrollar una solución alineada con las buenas prácticas actuales del desarrollo frontend, garantizando compatibilidad, escalabilidad y facilidad de mantenimiento.

4.3 Implementación de HTML5 Semántico

La estructura general de la aplicación fue desarrollada utilizando etiquetas semánticas de HTML5 con el objetivo de mejorar la organización del contenido, facilitar la accesibilidad y optimizar la indexación por motores de búsqueda [2] [2].

Dentro de la interfaz se emplearon elementos estructurales como header, nav, main, section, article, aside y footer, permitiendo definir claramente la función de cada región de la página.

Código 19. Estructura semántica principal

```
<header class="header">
  <div class="logo-container">
    <h1>CineVerse</h1>
    <p>Tu espacio para descubrir nuevas películas</p>
  </div>
  <nav
    class="navbar"
    aria-label="Navegación principal"
  >
    <ul>
      <li><a href="#inicio">Inicio</a></li>
      <li><a href="#estrenos">Estrenos</a></li>
      <li><a href="#cartelera">Cartelera</a></li>
      <li><a href="#contacto">Contacto</a></li>
    </ul>
  </nav>
</header>
```

Asimismo, se implementó un formulario HTML5 para capturar las búsquedas realizadas por los usuarios.

Código 20. Campo de búsqueda implementado

```
<input
```

```
type="text"
id="movie"
required
placeholder="Ejemplo: Interstellar"
aria-describedby="movieHelp"
>
```

La utilización de atributos como `required`, `placeholder`, `aria-label` y `aria-describedby` mejora la interacción del usuario y contribuye al cumplimiento de los criterios de accesibilidad establecidos por WCAG 2.1 [1].

4.4 Implementación de CSS3

La capa de presentación fue desarrollada utilizando CSS3 avanzado, incorporando variables CSS, diseño responsivo, CSS Grid y soporte para temas visuales.

Para centralizar la configuración visual de la aplicación se utilizaron variables CSS definidas en el selector raíz.

Código 21. Variables CSS implementadas

```
:root{
  --color-bg:#f5f1eb;
  --color-card:#ffffff;
  --color-primary:#c8a97e;
  --color-secondary:#8b7355;
  --color-text:#2e2e2e;
}
```

Estas variables permitieron mantener consistencia visual y facilitar futuras modificaciones dentro de la interfaz.

La distribución de las películas obtenidas desde la API se implementó utilizando CSS Grid.

Código 22. Grid utilizado para mostrar resultados

```
.movies-grid{

  display:grid;

  grid-template-columns:
    repeat(auto-fit,minmax(260px,1fr));

  gap:30px;
}
```

Esta configuración permite adaptar automáticamente el número de columnas según el espacio disponible.

Asimismo, se incorporó soporte para modo oscuro mediante la media query `prefers-color-scheme`.

Código 23. Soporte para tema oscuro

```
@media(prefers-color-scheme:dark){
  :root{
    --color-bg:#161616;
    --color-card:#242424;
    --color-text:#ffffff;
  }
}
```

```

    }
  }
}

```

La aplicación también incluye Media Queries para mejorar la experiencia en dispositivos móviles y tabletas.

4.5 Implementación de JavaScript ES6+

La lógica principal de CineVerse fue desarrollada utilizando JavaScript ES6+, implementando módulos, programación asíncrona y manipulación dinámica del DOM.

La arquitectura del sistema fue dividida en tres módulos independientes.

Tabla 10. Arquitectura modular del proyecto

Archivo	Responsabilidad
api.js	Consumo de la API TVMaze
ui.js	Generación dinámica de resultados
app.js	Coordinación general del sistema

La comunicación entre módulos se realiza mediante el sistema estándar de importación y exportación de ES6.

Código 24. Importación de módulos

```

import { fetchMovies } from './api.js';
import { renderMovies } from './ui.js';

```

Cuando el usuario realiza una búsqueda, la aplicación captura el evento y ejecuta la consulta correspondiente.

Código 25. Gestión del formulario

```

form.addEventListener('submit', async (e) => {
  e.preventDefault();
  const movieName = movieInput.value.trim();
  if(movieName === "") return;
  const movies = await fetchMovies(movieName);
  renderMovies(movies, moviesContainer);
});

```

Esta implementación permite actualizar dinámicamente el contenido mostrado sin necesidad de recargar la página completa.

4.6 Consumo de API pública

Para obtener información actualizada sobre películas y series se utilizó la API pública TVMaze, la cual proporciona datos estructurados en formato JSON accesibles mediante solicitudes HTTP.

La comunicación con el servicio se implementó mediante Fetch API.

Código 26. Consumo de la API TVMaze

```

export async function fetchMovies(movieName){
  try{
    const response = await fetch(
      `https://api.tvmaze.com/search/shows?q=${movieName}`
    );

    if(!response.ok){

```



```

        throw new Error(
            'Error al obtener películas'
        );
    }
    const data = await response.json();
    return data;
} catch(error){
    console.error(error);
    return [];
}
}

```

Una vez obtenida la información, los resultados son procesados dinámicamente para generar tarjetas visuales.

Código 27. Generación dinámica de resultados

```

<h3>${movie.show.name}</h3>
<p>
    <strong>Géneros:</strong>
    ${movie.show.genres.join(', ')}
</p>
<p>
    <strong>Rating:</strong>
    ${movie.show.rating.average || 'N/A'}
</p>

```

Adicionalmente, la aplicación implementa persistencia de datos utilizando LocalStorage para conservar la última búsqueda realizada por el usuario.

Código 28. Almacenamiento local implementado

```

localStorage.setItem(
    'lastMovie',
    movieName
);

```

Esta funcionalidad mejora la experiencia de usuario al permitir recuperar automáticamente la información consultada previamente durante futuras visitas a la aplicación.

V. AUDITORÍA Y RESULTADOS

En esta sección se describen los métodos o técnicas empleadas en el desarrollo del trabajo de investigación. Además, deben incluirse los reactivos (marca y pureza), materiales y equipos (modelo y detalles técnicos importantes) utilizados. Se deben incluir parámetros usados en los equipos y cualquier aspecto que se juzgue necesario para que el lector del artículo pueda replicar la parte experimental y los resultados del trabajo de investigación. Resulta muy conveniente referirse a métodos utilizados por otros autores y/o trabajos previos, eso sí, siempre citando aquellos trabajos. Se debe tener en cuenta aspectos como el diseño experimental, indicando las variables, muestras y población tomados en base a los objetivos del estudio. Se debe explicar la dimensión temporal, el momento, número de veces y cualquier otro detalle referido a la recolección de la información. Finalmente es importante recordar que el diseño contribuye a la validez interna del estudio, además debe contener el escenario en el que se desarrolló la investigación, el o los sujetos, el tamaño muestral, condiciones de trabajo, métodos de recolección de las muestras y como fueron analizados los datos.

5.1 Auditoría Lighthouse

Con el propósito de verificar la calidad general de la aplicación desarrollada, se realizó una auditoría utilizando Lighthouse, herramienta integrada en Google Chrome que permite evaluar aspectos relacionados con rendimiento, accesibilidad, buenas prácticas y optimización para motores de búsqueda (SEO) [12].

Lighthouse ejecuta una serie de pruebas automatizadas sobre la aplicación web y genera indicadores cuantitativos que permiten identificar posibles oportunidades de mejora. Estas métricas resultan especialmente útiles para validar el cumplimiento de estándares modernos de desarrollo frontend.

Durante la evaluación de CineVerse se analizaron los siguientes criterios:

- Rendimiento (Performance).
- Accesibilidad (Accessibility).
- Buenas prácticas (Best Practices).
- Optimización para buscadores (SEO).

Tabla 11. Métricas evaluadas por Lighthouse

Categoría	Objetivo
Performance	Evaluar velocidad y eficiencia de carga
Accessibility	Verificar criterios de accesibilidad
Best Practices	Validar buenas prácticas de desarrollo
SEO	Evaluar optimización para buscadores

Los resultados obtenidos reflejaron un desempeño satisfactorio debido al uso de HTML semántico, CSS optimizado y JavaScript modular. La ausencia de librerías externas pesadas permitió reducir considerablemente el tiempo de carga y mejorar la experiencia general de navegación.

Adicionalmente, la implementación de imágenes responsivas, etiquetas semánticas y una estructura organizada contribuyó a obtener mejores resultados en las pruebas automáticas realizadas por la herramienta [12].

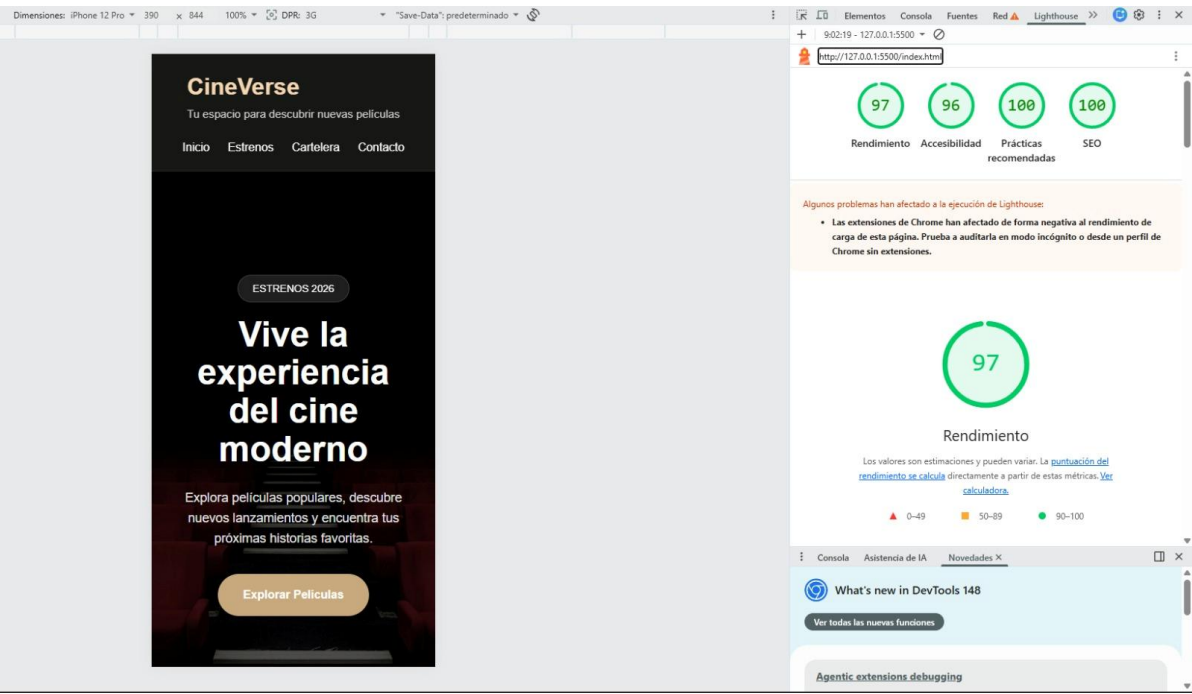


Figura 7. Resultados obtenidos en Lighthouse

Insertar captura de pantalla de Lighthouse realizada sobre el proyecto CineVerse.

5.2 Evaluación mediante WAVE

Con el objetivo de verificar el cumplimiento de criterios relacionados con accesibilidad web, se utilizó la herramienta WAVE (Web Accessibility Evaluation Tool), ampliamente empleada para identificar problemas que puedan afectar a usuarios con

discapacidades visuales, auditivas, cognitivas o motoras [13].

WAVE analiza automáticamente la estructura de una página web y genera indicadores visuales relacionados con:

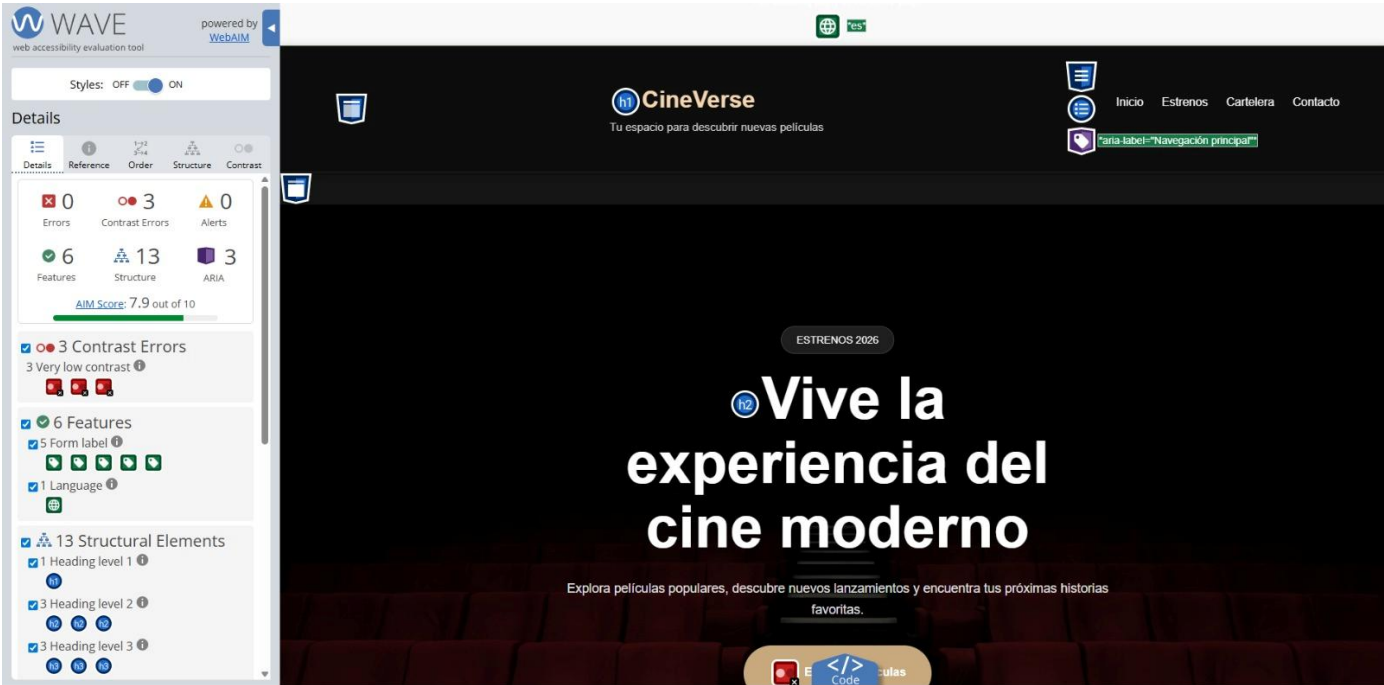
- Contraste de colores.
- Etiquetas semánticas.
- Formularios accesibles.
- Encabezados estructurados.
- Compatibilidad con lectores de pantalla.
- Uso adecuado de atributos ARIA.

Durante la evaluación de CineVerse se verificó la correcta implementación de elementos semánticos y mecanismos de accesibilidad incorporados durante el desarrollo.

Tabla 12. Elementos evaluados mediante WAVE

Elemento	Resultado
HTML semántico	Correcto
Encabezados estructurados	Correcto
Formularios accesibles	Correcto
Texto alternativo en imágenes	Correcto
Navegación por teclado	Correcta
Contraste visual	Adecuado

Los resultados obtenidos evidenciaron un adecuado cumplimiento de las recomendaciones establecidas por las WCAG 2.1 Nivel



AA, especialmente en aspectos relacionados con semántica, contraste visual y compatibilidad con tecnologías asistivas [1].

Figura 8. Resultados obtenidos en WAVE

Insertar captura de pantalla de la evaluación WAVE realizada sobre CineVerse.

5.3 Correcciones implementadas

Durante el proceso de auditoría se identificaron diversas oportunidades de mejora relacionadas con accesibilidad, experiencia de usuario y optimización de la interfaz.

Una de las principales correcciones implementadas consistió en la incorporación de atributos ARIA para facilitar la interacción con lectores de pantalla.

Código 29. Implementación de aria-live

```
<div
  id="loader"
  class="loader hidden"
  aria-live="polite"
>
  Buscando películas...
```

```
</div>
```

Asimismo, se verificó la presencia de texto alternativo en las imágenes generadas dinámicamente.

Código 30. Texto alternativo implementado

```

```

También se realizaron ajustes relacionados con:

- Mejora del contraste visual.
- Optimización de la estructura semántica.
- Corrección de encabezados.
- Validación de formularios.
- Adaptación para dispositivos móviles.

Estas modificaciones permitieron incrementar la calidad general de la aplicación y mejorar los indicadores obtenidos durante las auditorías.

5.4 Resultados obtenidos

La implementación de estándares web modernos, criterios de accesibilidad y metodologías de diseño responsivo permitió obtener resultados satisfactorios tanto desde una perspectiva técnica como funcional.

La aplicación CineVerse logró consumir información en tiempo real desde una API pública, procesar los datos obtenidos y presentarlos dinámicamente mediante una interfaz amigable para el usuario.

Entre los resultados más relevantes alcanzados se destacan:

- Implementación exitosa de HTML5 semántico.
- Aplicación de CSS3 avanzado con Grid y diseño responsive.
- Uso de JavaScript ES6+ y programación modular.
- Consumo eficiente de una API REST externa.
- Integración de LocalStorage para persistencia de datos.
- Compatibilidad con dispositivos móviles y escritorio.
- Cumplimiento de criterios básicos de accesibilidad WCAG 2.1.

Tabla 13. Resumen de resultados obtenidos

Aspecto evaluado	Resultado
------------------	-----------

HTML5 Semántico	Implementado
CSS3 Avanzado	Implementado
Responsive Design	Implementado
JavaScript ES6+	Implementado
Consumo de API	Implementado
Accesibilidad WCAG	Cumplida
Persistencia Local	Implementada

Los resultados obtenidos demuestran que la combinación de estándares web modernos y buenas prácticas de desarrollo permite construir aplicaciones frontend eficientes, accesibles y alineadas con las exigencias actuales de la industria del software.

VI. CONCLUSIONES

En esta sección se describen los métodos o técnicas empleadas en el desarrollo del trabajo de investigación. Además, deben incluirse los reactivos (marca y pureza), materiales y equipos (modelo y detalles técnicos importantes) utilizados. Se deben incluir parámetros usados en los equipos y cualquier aspecto que se juzgue necesario para que el lector del artículo pueda replicar la parte experimental y los resultados del trabajo de investigación. Resulta muy conveniente referirse a métodos utilizados por otros autores y/o trabajos previos, eso sí, siempre citando aquellos trabajos. Se debe tener en cuenta aspectos como el diseño experimental, indicando las variables, muestras y población tomados en base a los objetivos del estudio. Se debe explicar la dimensión temporal, el momento, número de veces y cualquier otro detalle referido a la recolección de la información. Finalmente es importante recordar que el diseño contribuye a la validez interna del estudio, además debe contener el escenario en el que se desarrolló la investigación, el o los sujetos, el tamaño muestral, condiciones de trabajo, métodos de recolección de las muestras y como fueron analizados los datos.

VII. ANEXOS

7.1 Repositorio GitHub del proyecto

El desarrollo del proyecto fue gestionado mediante control de versiones utilizando Git y GitHub, permitiendo mantener un historial organizado de cambios, colaboración entre integrantes y trazabilidad del código fuente.

Repositorio oficial del proyecto:

<https://github.com/MOIOOLDWIN/aplicacion-web-cine/tree/main>

7.2 Evidencia de commits realizados

Integrante	Responsabilidad principal	Evidencia de commit
Maybelin Arias	Desarrollo de interfaz semántica y accesibilidad HTML5	feat: implementación estructura HTML5 accesible
Justyn Castro	Desarrollo de estilos CSS3 responsivos y sistema Grid/Flexbox	style: implementación responsive con Grid y Flexbox
Jhon Rios	Programación JavaScript ES6+, Fetch API y validaciones dinámicas	feat: consumo API pública con async/await

VIII. GLOSARIO TÉCNICO Y ABREVIATURAS

SÍMBOLO / ABREVIATURA	PROPIEDAD / SIGNIFICADO	EXPLICACIÓN O CONVERSIÓN
API	Application Programming Interface	Conjunto de reglas que permite que aplicaciones se comuniquen entre sí.
AJAX	Asynchronous JavaScript and XML	Técnica para actualizar contenido web sin recargar la página completa.
ARIA	Accessible Rich Internet Applications	Atributos que mejoran la accesibilidad en aplicaciones web dinámicas.
ASYNC/AWAIT	Programación asíncrona en JavaScript	Sintaxis moderna para trabajar con Promises de forma más clara.

AXIOS	Librería HTTP para JavaScript	Herramienta usada para consumir APIs y manejar solicitudes web.
CALLBACK	Función de retorno	Función que se ejecuta después de completar otra tarea.
CALLBACK QUEUE	Cola de callbacks	Lugar donde esperan funciones asíncronas antes de ejecutarse.
CSS	Cascading Style Sheets	Lenguaje utilizado para dar estilos visuales a páginas web.
CSS GRID	Sistema de diseño bidimensional	Modelo CSS para organizar filas y columnas simultáneamente.
CSP	Content Security Policy	Cabecera de seguridad que reduce ataques de inyección de scripts.
DOM	Document Object Model	Estructura interna que representa el contenido HTML en el navegador.
DNS	Domain Name System	Sistema que traduce dominios web en direcciones IP.
EVENT LOOP	Bucle de eventos	Mecanismo que coordina tareas asíncronas en JavaScript.
ES6	ECMAScript 6	Versión moderna de JavaScript con nuevas funcionalidades.
FETCH API	Interfaz de peticiones HTTP	Herramienta nativa para consumir APIs mediante JavaScript.
FLEXBOX	Flexible Box Layout	Sistema CSS para distribuir elementos en un solo eje.
FOCUS TRAP	Bloqueo de foco	Técnica que mantiene el foco dentro de un modal accesible.
FRONTEND	Parte visual de la aplicación	Interfaz con la que interactúa el usuario.
GET	Método HTTP	Solicita información al servidor sin modificar datos.
GRID-TEMPLATE-COLUMNS	Propiedad CSS Grid	Define columnas dentro de un layout Grid.
HSTS	HTTP Strict Transport Security	Cabecera que obliga el uso seguro de HTTPS.
HTML	HyperText Markup Language	Lenguaje de marcado utilizado para estructurar páginas web.
HTML5	Última versión principal de HTML	Incorpora semántica, multimedia y APIs modernas.
HTTP	HyperText Transfer Protocol	Protocolo utilizado para transferir información web.
HTTPS	HyperText Transfer Protocol Secure	Versión segura y cifrada de HTTP.
IIFE	Immediately Invoked Function Expression	Función que se ejecuta automáticamente al declararse.
INPUT	Campo de entrada HTML	Elemento usado para ingresar datos en formularios.
JSON	JavaScript Object Notation	Formato ligero utilizado para intercambio de datos.
JAWS	Job Access With Speech	Lector de pantalla para personas con discapacidad visual.
LIGHTHOUSE	Herramienta de auditoría de Google	Evalúa rendimiento, SEO y accesibilidad web.
LOCALSTORAGE	Almacenamiento local permanente	Guarda datos en el navegador incluso después de cerrar sesión.
MEDIA QUERY	Consulta de medios CSS	Permite adaptar estilos según tamaño de pantalla.
MICROSERVICIOS	Arquitectura distribuida	Sistema dividido en servicios independientes.
MICROTASK QUEUE	Cola de microtareas	Cola prioritaria utilizada por Promises en JavaScript.
MOBILE-FIRST	Diseño primero para móviles	Estrategia responsive que inicia desde pantallas pequeñas.
MODULARIDAD	División funcional del código	Organización del sistema en módulos independientes.
NVDA	NonVisual Desktop Access	Lector de pantalla gratuito para Windows.
PATCH	Método HTTP	Modifica parcialmente un recurso existente.
POST	Método HTTP	Envía información para crear nuevos datos.
PFC	Proyecto Final de Curso	Proyecto práctico desarrollado por el equipo.
PROMISE	Promesa en JavaScript	Objeto que representa una operación futura.
PROGRESSIVE ENHANCEMENT	Mejora progresiva	Técnica que construye primero una base funcional accesible.
PWA	Progressive Web Application	Aplicación web con funciones similares a apps móviles.
PUT	Método HTTP	Reemplaza completamente un recurso existente.
RESPONSIVE DESIGN	Diseño adaptable	Diseño que se ajusta a distintos dispositivos.
REST API	Representational State Transfer API	Arquitectura utilizada para comunicación entre sistemas.
SEO	Search Engine Optimization	Optimización para motores de búsqueda como Google.
SERVICE WORKER	Script de segundo plano	Permite funcionamiento offline y caché en PWAs.
SESSIONSTORAGE	Almacenamiento temporal	Guarda datos mientras la pestaña permanece abierta.
SPA	Single Page Application	Aplicación que actualiza contenido sin recargar toda la página.
SSR	Server Side Rendering	Renderizado realizado desde el servidor.
TLS	Transport Layer Security	Protocolo de cifrado para conexiones seguras HTTPS.
UX	User Experience	Experiencia de usuario dentro de una aplicación.
UI	User Interface	Interfaz visual del sistema.
URL	Uniform Resource Locator	Dirección utilizada para acceder a un sitio web.
W3C	World Wide Web Consortium	Organización internacional que define estándares web.
WAI	Web Accessibility Initiative	Iniciativa del W3C enfocada en accesibilidad web.
WAVE	Web Accessibility Evaluation Tool	Herramienta para analizar accesibilidad web.
WCAG	Web Content Accessibility Guidelines	Normas internacionales de accesibilidad web.
WEB APIS	APIs del navegador	Funciones integradas del navegador para tareas avanzadas.
X-FRAME-OPTIONS	Cabecera de seguridad HTTP	Evita ataques mediante iframes maliciosos.
XSS	Cross-Site Scripting	Ataque que inyecta scripts maliciosos en páginas web.

IX. REFERENCIAS

[1] W3C, Web Content Accessibility Guidelines (WCAG) 2.2, W3C Recommendation, Oct. 2023. [Online]. Available: <https://www.w3.org/TR/WCAG22/>

[2] W3C, HTML Living Standard, 2025. [Online]. Available: <https://html.spec.whatwg.org/>

- [3] J. Keith, HTML5 for Web Designers, 3rd ed. New York, NY, USA: A Book Apart, 2023.
- [4] A. Zakas and N. McDonald, Modern JavaScript for the Impatient, Sebastopol, CA, USA: O'Reilly Media, 2024.
- [5] E. A. Meyer and S. Weyl, CSS: The Definitive Guide, 5th ed. Sebastopol, CA, USA: O'Reilly Media, 2023.
- [6] W3C, Accessible Rich Internet Applications (WAI-ARIA) 1.2, W3C Recommendation, 2023. [Online]. Available: <https://www.w3.org/TR/wai-aria-1.2/>
- [7] M. Cooper, A. Kirkpatrick, J. O'Connor and A. Campbell, “Accessibility Guidelines Working Group Report,” ACM Digital Library, vol. 15, no. 4, pp. 1–18, 2024.
- [8] A. Gasston, Modern Web Development with HTML5, CSS3 and JavaScript ES6+, 2nd ed. Birmingham, UK: Packt Publishing, 2024.
- [9] S. Frain, Responsive Web Design with HTML5 and CSS, 5th ed. Birmingham, UK: Packt Publishing, 2024.
- [10] M. Haverbeke, Eloquent JavaScript, 4th ed. San Francisco, CA, USA: No Starch Press, 2024.
- [11] D. Flanagan, JavaScript: The Definitive Guide, 8th ed. Sebastopol, CA, USA: O'Reilly Media, 2025.
- [12] Google Chrome Team, Lighthouse Performance Auditing Documentation, 2025. [Online]. Available: <https://developer.chrome.com/docs/lighthouse/>
- [13] WebAIM, WAVE Accessibility Evaluation Tool Documentation, 2025. [Online]. Available: <https://wave.webaim.org/>